

Name: Varanasi L V S S S Sathveek
Roll no: 423189

Circular Queue

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define cap 10
```

```
struct queue {
    int arr[cap];
    int front, rear;
};
```

```
typedef struct queue que;
```

```
que* create() {
    que *q = (que *)malloc(sizeof(que));
    if (!q) {
        return NULL;
    }
    q->front = -1;
    q->rear = -1;
    return q;
}
```

```
int destroy(que *q) {
    if (!q) {
        return 0;
    }
    free(q);
    return 1;
}
```

```
int isFull(que *q) {
    return ((q->rear + 1) % cap == q->front) ? 1 : 0;
}
```

```
int isEmpty(que *q) {
    return (q->front == -1) ? 1 : 0;
}
```

```
int enqueue(que *q, int data) {
    if (isFull(q)) {
```

```

        return 0;
    }
    if (isEmpty(q)) {
        q->front = 0;
    }
    q->rear = (q->rear + 1) % cap;
    q->arr[q->rear] = data;
    return 1;
}

int dequeue(que *q, int *ptr) {
    if (isEmpty(q)) {
        return 0;
    }
    *ptr = q->arr[q->front];
    if (q->front == q->rear) {
        q->front = -1;
        q->rear = -1;
    } else {
        q->front = (q->front + 1) % cap;
    }
    return 1;
}

int main() {
    que *q = create();
    int value;

    enqueue(q, 1);
    enqueue(q, 2);
    enqueue(q, 3);

    while (!isEmpty(q)) {
        dequeue(q, &value);
        printf("%d\n", value);
    }

    destroy(q);
    return 0;
}

```

Output

```

1
2
3

```

DEqueue(with no restrictions)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define CAP 10
```

```
struct deque {  
    int arr[CAP];  
    int front, rear;  
};
```

```
typedef struct deque deque;
```

```
deque* create() {  
    deque *d = (deque *)malloc(sizeof(deque));  
    if (!d) {  
        return NULL;  
    }  
    d->front = -1;  
    d->rear = -1;  
    return d;  
}
```

```
int destroy(deque *d) {  
    if (!d) {  
        return 0;  
    }  
    free(d);  
    return 1;  
}
```

```
int isFull(deque *d) {  
    return ((d->rear + 1) % CAP == d->front) ? 1 : 0;  
}
```

```
int isEmpty(deque *d) {  
    return (d->front == -1);  
}
```

```
int enqueueFront(deque *d, int data) {  
    if (isFull(d)) {  
        return 0;  
    }  
    if (isEmpty(d)) {
```

```

        d->front = 0;
        d->rear = 0;
    } else {
        d->front = (d->front - 1 + CAP) % CAP;
    }
    d->arr[d->front] = data;
    return 1;
}

```

```

int enqueueRear(deque *d, int data) {
    if (isFull(d)) {
        return 0;
    }
    if (isEmpty(d)) {
        d->front = 0;
        d->rear = 0;
    } else {
        d->rear = (d->rear + 1) % CAP;
    }
    d->arr[d->rear] = data;
    return 1;
}

```

```

int dequeueFront(deque *d, int *ptr) {
    if (isEmpty(d)) {
        return 0;
    }
    *ptr = d->arr[d->front];
    if (d->front == d->rear) {
        d->front = -1;
        d->rear = -1;
    } else {
        d->front = (d->front + 1) % CAP;
    }
    return 1;
}

```

```

int dequeueRear(deque *d, int *ptr) {
    if (isEmpty(d)) {
        return 0;
    }
    *ptr = d->arr[d->rear];
    if (d->front == d->rear) {
        d->front = -1;
    }
}

```

```

        d->rear = -1;
    } else {
        d->rear = (d->rear - 1 + CAP) % CAP;
    }
    return 1;
}

int main() {
    deque *d = create();
    int value;

    enqueueRear(d, 1);
    enqueueRear(d, 2);
    enqueueRear(d, 3);

    while (!isEmpty(d)) {
        dequeueFront(d, &value);
        printf("Dequeued from front: %d\n", value);
    }

    enqueueFront(d, 4);
    enqueueFront(d, 5);
    enqueueFront(d, 6);

    while (!isEmpty(d)) {
        dequeueRear(d, &value);
        printf("Dequeued from rear: %d\n", value);
    }

    destroy(d);
    return 0;
}

```

Output

```

Dequeued from front: 1
Dequeued from front: 2
Dequeued from front: 3
Dequeued from rear: 4
Dequeued from rear: 5
Dequeued from rear: 6

```